

第 8 章 项目管理

在现代项目实践中，由于项目管理在项目实施过程中发挥重大作用，其已经成为每个项目必不可少的工作内容。项目管理是保证项目成功的核心手段，成熟的项目管理体系是项目经理的得力工具。因此，作为信息系统项目中的骨干成员，系统分析师必须熟悉项目管理知识，掌握项目管理的主要过程、方法、技术和工具。

8.1 项目管理概述

项目是在特定条件下，具有特定目标的一次性任务，是在一定时间内，满足一系列特定目标的多项相关工作的总称。从项目的基本定义出发，人们日常进行的所有活动都可定义或分解为项目。

项目管理是管理学的一个分支学科，是指在项目活动中运用专门的知识、技能、工具和方法，使项目能够在有限资源限定条件下，实现或超过设定的需求和期望的过程。项目管理是对一些成功达成一系列目标相关活动（譬如任务）的整体监测和管控，包括策划、进度计划和维护组成项目的活动的进展。

软件项目管理的对象是软件工程项目。它所涉及的范围覆盖了整个软件工程过程。为使软件项目开发获得成功，关键问题是必须对软件项目的工作范围、可能风险、需要资源（人、硬件/软件）、要实现的任务、经历的里程碑、花费工作量（成本）、进度安排等做到心中有数。这种管理在技术工作开始之前开始，在软件从概念到实现的过程中继续进行，当软件工程过程最后结束时才终止。

软件项目管理是为了使软件项目能够按照预定的成本、进度、质量顺利完成，而对人员（People）、产品（Product）、过程（Process）和项目（Project）等进行分析和管理的活动。

8.2 范围管理

范围是项目目标的更具体的表达。在信息系统项目实践中，需求无序扩张是项目失败最常见的原因之一，往往在项目启动、计划、执行，甚至收尾时还在不断地加入新的功能。无论是客户的要求，还是项目团队成员对新技术的试验，都可能导致项目范围的失控，从而使项目的时间、资源和质量上都受到严重影响。

范围管理就是要确定项目的边界，也就是说，要确定哪些工作是本次项目应该做的，哪些工作不应该包括在本次项目中。这个过程用于确保项目干系人对作为项目结果的产品（或服务），以及开发这些产品所确定的过程有一个共同的理解。

8.2.1 范围计划的编制

项目范围对项目的影响是决定性的，只有完成了项目范围中的全部工作，项目才能结束。因此，一个范围不明确或干系人对范围理解不一致的项目不可能获得成功。范围不明确最可能的后果是项目的范围蔓延，项目永远也做不到头；对范围的理解不一致的结果往往是项目组的工作无法得到其他干系人的认可。要防止这些事件的发生，首要任务就是要编制好范围管理计划。

范围计划编制的成果就是范围管理计划。范围管理计划是对项目的范围进行确定、记载、核实管理和控制的行动指南，它与项目范围计划不同，范围计划描述的是项目的边界，而范围管理计划描述的是项目组将如何进行项目的范围管理。具体来说，范围管理计划包括如何进行项目范围定义，如何制订工作分解结构（Work Breakdown Structure, WBS），如何进行项目范围核实和控制等。范围管理计划应该对怎样变化、变化频率如何，以及变化了多少等项目范围预期的稳定性进行充分的评估。范围管理计划也应该包括对变化范围怎样确定，变化应归为哪一类等问题的清楚描述。事实上，在项目的产品范围还没有确定之前，确定这些问题是非常困难的，但是仍然有必要进行。

8.2.2 创建工作分解结构

WBS 把项目整体或者主要的可交付成果分解成容易管理、方便控制的若干个子项目，子项目需要继续分解为工作包。持续这个过程，直到整个项目都分解为可管理的工作包，这些工作包的总和就是项目的所有工作范围。

创建 WBS 的目的是详细规定项目的范围，建立范围基准。WBS 是面向可交付物的项目元素的层次分解，它组织并定义了整个项目范围。当一个项目的 WBS 分解完成后，项目相关人员对完成的 WBS 应该给予确认，并对此达成共识。然后，才能据此进行时间和成本的估算。

在每个分解单元中，都存在可交付成果和里程碑。里程碑标志着某个可交付成果或者阶段的正式完成。里程碑和可交付成果紧密联系在一起，但并不是一个事物。可交付成果可能包括报告、原型、成果和最终系统。而里程碑则关注是否完成，例如，正式的用户认可文件。WBS 中的任务有明确的开始时间和结束时间，任务的结果可以和预期的结果相比较。

最底层的工作单元称为工作包，它应该便于完整地分派给不同的人或组织，所以要求明确各工作单元直接的界面。工作包应该非常具体，以便承担者能明确自己的任务、努力的目标和承担的责任，工作包是基层任务或工作的指派，同时其具有检测和报告工作的作用。所有工作包的描述必然让成本会计管理者和项目监管人员理解，并能够清楚地区分不同工作包的工作。同时，工作包的大小也是需要考虑的细节，如果工作包太大，则难以达到可管理和可控制的目标；如果工作包太小，则 WBS 就要消耗项目团队成员的大量时间和精力。

实际使用中，创建 WBS 没有所谓的“正确的”方式，既可以使用白板、草图等，也可以使用专门的项目管理软件，如 Microsoft Project 等。

8.2.3 范围确认和控制

在对项目范围进行了定义，编制了详细的 WBS 之后，接下来的事情就是要让项目干系人

确认范围，以及在项目实施过程中，对范围进行控制，确保项目范围按照既定流程进行变更。

1. 范围确认

范围确认主要是确认项目的可交付成果是否满足项目干系人的要求。把项目的可交付成果列表提交给项目干系人，同时，也应该展示项目的进度安排。项目干系人进行范围确认时，检查内容主要包括：

(1) 可交付成果是否是确定的、可核实的。

(2) 每个可交付成果是否有明确的里程碑，里程碑是否有明确的、可辨别的事件，例如，客户的书面认可。

(3) 是否有明确的质量标准，也就是说，可交付成果的交付要有是否按照要求完成的标准，可交付成果和其标准之间是否有明确的联系。

(4) 审核和承诺是否有清晰的表达。项目投资人必须正式地同意项目的边界、项目完成的产品（或服务），以及项目相关的可交付成果。项目组必须清楚地了解可交付成果是什么。所有这些表达必须清晰，并取得一致同意。

(5) 项目范围是否覆盖了需要完成的产品（或服务）进行的所有活动，有没有遗漏或者错误。

(6) 项目范围的风险是否太高，管理层是否能够降低可预见的风险发生时对项目的冲击。

在范围确认的过程中，如果发现项目范围说明书、WBS 中有遗漏或者错误，需要向项目组明确指出错误的内容，并给出修正意见，项目组需要根据这些意见修改相关文档。在范围确认的过程中，也可能出现范围变更请求，如果这些请求得到了批准，那么，也要修改相关文档。

2. 范围控制

在信息系统项目的实施过程中，项目范围难免会因为各种因素而发生变更。例如，项目外部环境发生变化（例如，法律、对手的新产品等）；范围计划不周，有错误或者遗漏；出现了新的技术、手段和方案；项目实施组织发生了变化；客户对项目或者项目产品的要求发生了变化等。所有的这些变化，即使是“好”的变化，对项目管理人员而言，也会令人不安。

项目范围定义了项目应该做的和不应该做的，那么对于范围变更，就不能随意进行。对于用户不断提出的新要求和建议，项目组应该坚持“决不让步，除非交换”的原则，尽可能减少范围蔓延的可能性，所有的范围变更必须在项目的工期、费用或者质量要求上得到相应的变更。

所有的变更必须记载，范围控制必须能够对造成范围变更的因素施加影响，估算对项目的资金、进度和风险等的影响，以保证变化是有利的。同时，需要判断范围变更是否发生，如果已经发生，则要对变更进行管理。

对范围变更进行控制时，要以 WBS、项目进展报告、变更请求和范围管理计划为依据。对于有合同的项目而言，项目范围变更必须遵守项目合同的相关条款。

8.3 进度管理

根据项目的定义，项目是在一定的时间内完成其目标的一次性任务。因此，能否在给定的

时间内交付产品（或服务）是衡量项目是否成功的重要标志。进度管理就是采用科学的方法，确定进度目标，编制进度计划和资源供应计划，进行进度控制，在与质量、成本目标协调的基础上，实现工期目标。具体来说，进度管理包括以下过程：

- （1）活动定义。确定完成项目各项可交付成果而需要开展的具体活动。
- （2）活动排序。识别和记录各项活动之间的先后关系和逻辑关系。
- （3）活动资源估算。估算完成各项活动所需要的资源类型和数量。
- （4）活动历时估算。估算完成各项活动所需要的时间。
- （5）进度计划编制。分析活动顺序、活动持续时间、资源要求和进度制约因素，制订项目进度计划。
- （6）进度控制。根据进度计划开展项目活动，如果发现偏差，则分析原因或进行调整。

8.3.1 活动排序

在项目中，一个活动的执行可能需要依赖于另外一些活动的完成，也就是说，它的执行必须在某些活动完成之后，这就是活动的先后依赖关系。一般来说，依赖关系的确定应首先分析活动之间本身存在的逻辑关系，在此逻辑关系的基础上再加以充分分析，以确定各活动之间的组织关系，这就是活动排序。

1. 前导图法

前导图法（Precedence Diagramming Method, PDM）用方格或矩形（节点）表示活动，用箭线表示依赖关系。在 PDM 中，每项活动都有唯一的活动号，注明了预计工期。每个节点的活动有最早开始时间（Early Start, ES）、最迟开始时间（Late Start, LS）、最早结束时间（Early Finish, EF）和最迟结束时间（Late Finish, LF）。PDM 节点的几种表示方法如图 8-1 所示。

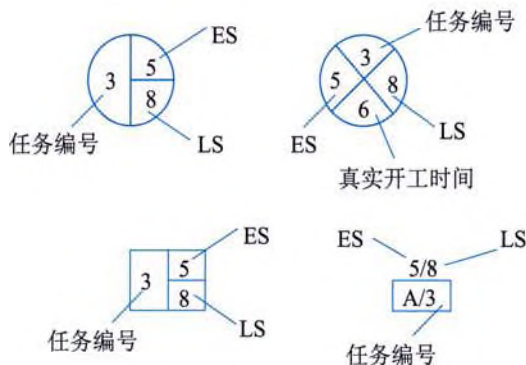


图 8-1 节点表示法

PDM 包括 4 种依赖关系或先后关系：

- （1）完成对开始（FS）。后一活动的开始要等到前一活动的完成。
- （2）完成对完成（FF）。后一活动的完成要等到前一活动的完成。
- （3）开始对开始（SS）。后一活动的开始要等到前一活动的开始。
- （4）开始对完成（SF）。后一活动的完成要等到前一活动的开始。

以上4种关系的表示如图8-2所示。

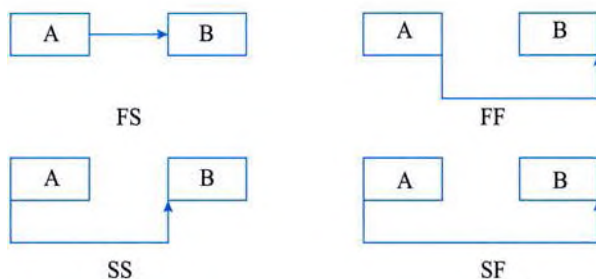


图8-2 活动依赖关系图

在PDM中,FS是最常用的逻辑关系,而SF关系很少用。在绘制PDM时,需要遵守下列规则:

- (1) 必须正确表达项目中活动之间的逻辑关系,图中不能出现回路。
- (2) 图中不能出现双向箭头或无箭头的连线,不能出现无箭尾节点的箭线或无箭头节点的箭线。
- (3) 图中只能有一个起始节点和一个终止节点。当图中出现多项无内向箭线的活动或多项无外向箭线的活动时,应在PDM的开始或者结束处设置一项虚活动(不占用任何资源,只表示逻辑关系的活动),作为该PDM的起始节点或终止节点。

2. 箭线图法

箭线图法(Arrow Diagramming Method, ADM)用节点表示事件,用箭线表示活动,并在节点处将其连接起来,以表示依赖关系。在ADM中,给每个事件而不是每项活动指定一个唯一的号码。活动的开始(箭尾)事件叫做该活动的紧前事件(Precede Event),活动的结束(箭头)事件叫做该活动的紧后事件(Successor Event)。在ADM中,有三个基本原则:

- (1) 每一个事件必须有唯一的代号,即ADM中不会有相同的代号。
- (2) 任何两项活动的紧前事件和紧后事件代号至少有一个不相同,节点序号沿箭线方向越来越大。
- (3) 流入(流出)同一事件的活动,均有共同的后继活动(或先行活动)。

ADM只使用FS关系,因此可能要使用虚活动才能正确地定义所有的逻辑关系,虚活动用虚箭线表示。在复杂的ADM中,为避免多个起点或终点引起的混淆,也可以用虚活动来解决,如图8-3所示。

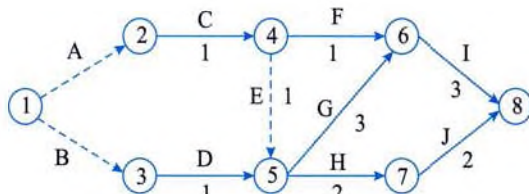


图8-3 箭线图示例

3. 确定依赖关系

在项目进度管理中，通常使用三种依赖关系来进行活动排序，分别是强制性依赖关系、可自由处理的依赖关系和外部依赖关系。

(1) 强制性依赖关系，也称为硬逻辑关系、工艺关系。这是活动固有的依赖关系，这种关系是活动之间本身存在的、无法改变的逻辑关系。

(2) 可自由处理的依赖关系，也称为软逻辑关系、组织关系、首选逻辑关系、优先逻辑关系。这是人为确定的一种先后关系。

(3) 外部依赖关系。这种关系涉及项目与非项目活动之间的关系。

逻辑关系的表达可以分为平行、顺序和搭接三种形式。

(1) 平行关系，也称为并行关系。两项活动同时开始即为平行关系。例如，在图 8-3 中，活动 A 和 B 是平行关系。

(2) 顺序关系。相邻两项活动先后进行即为顺序关系。如前一活动完成后，后一活动马上开始则为紧连顺序关系。如后一活动在上一活动完成后隔一段时间才开始则为间隔顺序关系。在顺序关系中，当一项活动只有在另一项活动完成以后才能开始，并且中间不插入其他活动，则称另一项活动为该活动的紧前活动；反之，当一项活动只有在完成之后，另一项活动才能开始，并且中间不插入其他活动，则称另一项活动为该活动的紧后活动。例如，在图 8-3 中，活动 A 和 C 为紧连顺序关系，A 和 E 是间隔顺序关系，A 是 C 的紧前活动，C 是 A 的紧后活动。

(3) 搭接关系。两项活动只有一段时间是平行进行的则称为搭接关系。

8.3.2 活动资源估算

活动资源估算包括决定需要什么资源（例如，人员、工具、设备等）和资源数量，以及何时使用资源来有效地执行项目活动。它必须和成本估算相结合。

进行资源估算时，必须要对资源的可用性进行评价，否则，将只能是纸上谈兵，包括考虑资源地理位置的改变，以及资源数量和级别在项目进行过程中可能会改变。例如，在软件开发的前期，需要一些系统分析师的参与，而在编码阶段，则需要很多程序员的参与。

进行活动资源估算的方法主要有专家判断法、替换方案的确定、公开的估算数据、估算软件和自下而上的估算。

(1) 专家判断法。专家判断法通常是由项目成本管理专家根据以往类似项目的经验和对本项目的判断，经过周密思考，进行合理预测，从而估算出项目资源。进行预测的专家可以是任何具有专门知识或经过特别培训的组织和个人。

(2) 替换方案的确定。资源估算是为了给项目预算明确空间，为早期的资源筹备提供数据，如果某项活动存在替代方案，或提供的资源有替代支持可能，则需要明确声明。

(3) 公开的估算数据。有些公司会定期公开一些生产率或人工费率数据，其中包括很多国家和地区的劳动力交易、材料和设备信息。这些数据可以作为资源估算的参考。

(4) 估算软件。依靠软件的强大功能，可以定义资源可用性、费率，以及不同的资源日历。

(5) 自下而上的估算。把复杂的活动分解为更小的工作，以便于资源估算。将每项工作所需要的资源估算出来，然后汇总，即是整个活动所需要的资源数量。

8.3.3 活动历时估算

活动历时估算直接关系各项具体活动和完成整个项目所需要的总体时间的估算。活动历时估算通常要同时考虑间隔时间。在估算时,要在综合考虑各种资源、人力、物力、财力的情况下,对项目中各工作分别进行时间估计。若活动时间估算太短,则在工作中会出现被动紧张的局面;反之,如果活动时间估算太长,则会使整个项目的完工期限延长,从而造成损失。

1. 软件项目的工作量

软件项目的工作量和工期的估算比较复杂,因为软件本身的复杂性、历史经验的缺乏、估算工具的缺乏,以及一些人为错误,导致软件项目的规模估算往往和实际情况相差甚远。因此,估算错误是软件项目失败的主要原因之一。

软件项目通常用代码行(Line of Code, LOC)来衡量项目规模,LOC指所有可执行的源代码行数,包括可交付的工作控制语言语句、数据定义、数据类型声明、等价声明、输入输出格式声明等。项目经理可以根据对历史项目的审计来核算本企业的单行代码价值。例如,假设某公司每一万行Java语言源代码形成的源文件约为250kB,视频点播系统项目的源文件大小为3.75MB,则可估计该项目源代码大约为15万行,该项目累计投入工作量为240人·月,每人·月费用为10 000元(包括人均工资、福利、办公费用分摊等),则该项目中1LOC的价值为:

$$(240 \times 10\,000) / 150\,000 = 16 \text{ 元/LOC}$$

该项目的人·月平均代码行数为:

$$150\,000 / 240 = 625 \text{ LOC/人·月}$$

2. 德尔菲法

德尔菲(Delphi)法是当前比较流行的专家评估技术,该方法结合了专家判断法和三点估算法,在没有历史数据的情况下,对项目进行估算。德尔菲法对减少数据中人为的偏见、防止任何人对结果不适当地产生过大的影响尤其有用,其具体实施步骤如下:

(1) 组织者发给每位专家一份软件系统的规格说明书(略去名称和单位)和一张记录估算值的表格,请他们进行估算。

(2) 专家详细研究软件规格说明书的内容,对该软件提出三个规模的估算值,即:

a_i : 该软件可能的最小规模(最少源代码行数)。

m_i : 该软件最可能的规模(最可能的源代码行数)。

b_i : 该软件可能的最大规模(最多源代码行数)。

无记名地填写表格,并说明做此估算的理由。在填表的过程中,专家互相不进行讨论,但可以向组织者提问。

(3) 组织者为专家们填在表格中的数据进行整理,计算各位专家的估算期望值 E_i ,并综合各位专家估算值的期望中值 E (这种方法通常也称为三点估算法)。

$$E_i = \frac{a_i + 4m_i + b_i}{6} \quad E = \frac{1}{n} \sum_{i=1}^n E_i$$

(4) 将各位专家的第一次判断意见汇总,列成图表,进行对比,再分发给各位专家,让专

家比较自己与他人的不同意见，修改自己的估算和判断。然后比较两次估算的结果。若差异很大，则要通过查询找出差异的原因。

(5) 上述过程可重复多次。最终可获得一个得到多数专家共识的软件规模（源代码行数）。在此过程中不得进行小组讨论。在向专家进行反馈的时候，只给出各种意见，但并不说明发表各种意见的专家的具体姓名。

(6) 通过与历史资料进行类比，根据过去完成软件项目的规模和成本等信息，推算出该软件每行源代码所需要的成本。然后再乘以该软件源代码行数的估算值，就可得到该软件的成本估算值。

德尔菲法与常见的召集专家开会、通过集体讨论得出一致预测意见的专家会议法既有联系又有区别。德尔菲法能发挥专家会议法的优点，即能充分发挥各位专家的作用，集思广益，准确性高；能把各位专家意见的分歧点表达出来，取各家之长，避各家之短。同时，德尔菲法又能避免专家会议法的缺点，即权威人士的意见影响他人的意见；有些专家碍于情面，不愿意发表与其他人不同的意见；出于自尊心而不愿意修改自己原来不全面的意见。

德尔菲法的主要缺点是过程比较复杂，花费时间较长。另外，人们无法利用其他参加者的估算值来调整自己的估算值。宽带德尔菲法克服了这个缺点。在专家正式将估算值填入表格之前，由组织者召集小组会议，专家们与组织者一起对估算问题进行讨论，然后专家们再无记名填表。组织者对各位专家在表中填写的估算值进行综合和分类后，再召集会议，请专家们对其估算值有很大变动之处进行讨论，请专家们重新无记名填表。这样适当重复几次，得到比较准确的估计值。由于增加了协商的机会，集思广益，使得估算值更趋于合理。

3. 类比估算法

类比估算法适合评估一些与历史项目在应用领域、环境和复杂度等方面相似的项目，通过新项目与历史项目的比较得到规模估计。由于类比估算法估计结果的精确度取决于历史项目数据的完整性和准确度，因此，用好类比估算法的前提条件之一是企业建立起较好的项目后评价与分析机制，对历史项目的数据分析是可信赖的。

类比估算法的基本步骤如下：

- (1) 整理出项目功能列表和实现每个功能的代码行。
- (2) 标识出每个功能列表与历史项目的相同点和不同点，特别要注意历史项目做得不够的地方。
- (3) 通过步骤(1)和(2)得出各个功能的估计值。
- (4) 产生规模估计。

软件项目中用类比估算法，往往还要解决可复用代码的估算问题。估计可复用代码量的最好办法就是由程序员或系统分析师详细地考查已存在的代码，估算出新项目可复用的代码中需重新设计的代码百分比、需重新编码或修改的代码百分比，以及需重新测试的代码百分比。根据这三个百分比，可用下面的计算公式计算等价新代码行：

$$\text{等价 LOC} = [(\text{重新设计 \%} + \text{重新编码 \%} + \text{重新测试 \%})/3] \times \text{已有 LOC}$$

例如，有 10 000LOC，假定 30% 需要重新设计，50% 需要重新编码，70% 需要重新测试，

那么其等价的 LOC 可以计算为:

$$[(30\%+50\%+70\%)/3] \times 10\,000 = 5000\text{LOC}$$

即复用这 10 000LOC 相当于编写 5000LOC 的工作量。

4. 功能点估算法

功能点 (Function Point, FP) 估算是在需求分析阶段基于系统功能的一种规模估计方法。FP 估算并不是集中于功能上, 而是通过研究初始应用需求, 确定各种输入、输出、数据文件、查询和外部接口, 以及一些复杂度调整值。通常的步骤如下:

- (1) 计算输入、输出、查询、数据文件与接口的数目。
- (2) 将这些数据进行加权乘。
- (3) 根据对复杂度的判断, 总数可以用复杂性调节因子进行调整。

统计发现, 对一个软件产品的开发而言, 功能点对项目早期的规模估计很有帮助。功能点估算法与程序设计语言无关, 但涉及的主观因素比较多。例如, 各种权值函数的取值等。

5. COCOMO 模型

COCOMO 模型 (Constructive Cost Model) 是一种结构型估算模型, 是一种精确、易于使用的估算方法。在 COCOMO 模型中, 考虑开发环境, 软件开发项目的总体类型可分为三种: 组织型、嵌入型和半独立型。COCOMO 模型按其详细程度分成三级: 基本 COCOMO 模型、中间 COCOMO 模型和详细 COCOMO 模型。

(1) 基本 COCOMO 模型。基本 COCOMO 模型是一个静态单变量模型, 它用一个已估算出来的源代码行数为自变量的 (经验) 函数来计算软件开发工作量。具体计算公式如表 8-1 所示。其中, $k\text{LOC}=1000\text{LOC}$; M 表示开发工作量, 单位为人·月; T 表示开发进度, 单位为月。

表 8-1 基本 COCOMO 模型的工作量和进度公式

总体类型	工作量	进度
组织型	$M = 2.4 \times k\text{LOC}^{1.05}$	$T = 2.5 \times M^{0.38}$
半独立型	$M = 3.0 \times k\text{LOC}^{1.12}$	$T = 2.5 \times M^{0.35}$
嵌入型	$M = 3.6 \times k\text{LOC}^{1.20}$	$T = 2.5 \times M^{0.32}$

(2) 中间 COCOMO 模型。中间 COCOMO 模型则在用 LOC 为自变量的函数计算软件开发工作量 (此时称为名义工作量) 的基础上, 再用涉及产品、硬件、人员、项目等方面属性的影响因素来调整工作量的估算。具体计算公式如表 8-2 所示。其中 EAF 为调整因子, 取值范围为 0.9 ~ 1.4。

表 8-2 中间 COCOMO 模型的工作量和进度公式

总体类型	工作量	进度
组织型	$M = 3.2 \times k\text{LOC}^{1.05} \times \text{EAF}$	$T = 2.5 \times M^{0.38}$
半独立型	$M = 3.0 \times k\text{LOC}^{1.12} \times \text{EAF}$	$T = 2.5 \times M^{0.35}$
嵌入型	$M = 2.8 \times k\text{LOC}^{1.20} \times \text{EAF}$	$T = 2.5 \times M^{0.32}$

(3) 详细 COCOMO 模型。详细 COCOMO 模型包括中间 COCOMO 模型的所有特性，但用各种影响因素调整工作量估算时，还要考虑对软件工程过程中每一步骤（例如，分析、设计等）的影响。详细 COCOMO 模型的名义工作量公式和进度公式与中间 COCOMO 模型相同。但详细 COCOMO 模型分层、分阶段给出工作量因素分级表。针对每一个影响因素，按模块层、子系统层、系统层，有三张不同的工作量因素分级表，供不同层次的估算使用。

8.3.4 进度控制

项目进度控制就是将实际进度与计划进度进行比较并分析结果，以保持项目工期不变，保证项目质量和所耗费用最少为目标，做出有效对策，进行项目进度更新。项目进度更新主要包括两方面工作，即分析进度偏差的影响和进行项目进度计划的调整。这两方面的工作都是以关键路径及相关参数为基础，可能要使用挣值分析的结果。

1. 分析进度偏差的影响

当出现进度偏差时，需要分析该偏差对后续活动及总工期的影响。主要从以下几方面进行分析：

(1) 分析产生进度偏差的活动是否为关键活动。若出现偏差的活动是关键活动，则无论其偏差大小，对后续活动及总工期都会产生影响，必须进行进度计划更新；若出现偏差的活动为非关键活动，则需根据偏差值与总时差和自由时差（本活动总时差 - 紧后活动总时差的最小值）的大小关系，确定其对后续活动和总工期的影响程度。

(2) 分析进度偏差是否大于总时差。如果活动的进度偏差大于总时差，则必将影响后续活动和总工期，应采取相应的调整措施；若活动的进度偏差小于或等于该活动的总时差，表明对总工期无影响；但其对后续活动的影响，需要将其偏差与其自由时差相比较才能做出判断。

(3) 分析进度偏差是否大于自由时差。如果活动的进度偏差大于该活动的自由时差，则会对后续活动产生影响，如何调整应根据后续活动允许影响的程度而定；若活动的进度偏差小于或等于该活动的自由时差，则对后续活动无影响，进度计划可不进行调整更新。

经过上述分析，项目管理人员可以确定应该调整产生进度偏差的活动和调整偏差值的大小，以便确定应采取的调整更新措施，形成新的符合实际进度情况和计划目标的进度计划。

2. 项目进度计划的调整

项目进度计划的调整往往是一个持续反复的过程，一般分几种情况：

(1) 关键活动的调整。对于关键路径，由于其中任一活动持续时间的缩短或延长都会对整个项目工期产生影响。因此，关键活动的调整是项目进度更新的重点。有以下两种情况：

第一种情况：关键活动的实际进度较计划进度提前时的调整方法。若仅要求按计划工期执行，则可利用该机会降低资源强度及费用。实现的方法是，选择后续关键活动中资源消耗量大或直接费用高的予以适当延长，延长的时间不应超过已完成的关键活动提前的量；若要求缩短工期，则应将计划的未完成部分作为一个新的计划，重新计算与调整，按新的计划执行，并保证新的关键活动按新计算的时间完成。

第二种情况：关键活动的实际进度较计划进度落后时的调整方法。调整的目标就是采取措施将耽误的时间补回来，保证项目按期完成。调整的方法主要是缩短后续关键活动的持续时间。这种方法是指在原计划的基础上，采取组织措施或技术措施缩短后续工作的持续时间以弥补时间损失，确保总工期不延长。

实际上，不得不延长工期的情况非常普遍，在项目总计划的制订中要充分考虑适当的时间冗余。当预计到项目时间要拖延时应该分析原因，第一时间给项目干系人通报，并征求项目业主（建设单位）的意见，这也是项目进度控制的重要工作内容。

(2) 非关键活动的调整。当非关键路径上某些工作的持续时间延长，但不超过其时差范围时，则不会影响项目工期，进度计划不必调整。为了更充分地利用资源，降低成本，必要时可对非关键活动的时差做适当调整，但不得超出总时差，且每次调整均需进行时间参数计算，以观察每次调整对计划的影响。

非关键活动的调整方法有三种：在总时差范围内延长非关键活动的持续时间、缩短工作的持续时间、调整工作的开始或完成时间。当非关键路径上某些工作的持续时间延长而超出总时差范围时，则必然影响整个项目工期，关键路径就会转移。这时，其调整方法与“关键活动的调整”方法相同。

(3) 增减工作项目。由于编制计划时考虑不周，或因某些原因需要增加或取消某些工作，则需重新调整网络计划，计算网络参数。由于增减工作项目不应影响原计划总的逻辑关系，以便使原计划得以实施。因此，增减工作项目，只能改变局部的逻辑关系。

增加工作项目，只对原遗漏或不具体的逻辑关系进行补充；减少工作项目，只是对提前完成的工作项目或原不应设置的工作项目予以消除。增减工作项目后，应重新计算网络时间参数，以分析此项调整是否对原计划工期产生影响，若有影响，应采取措施使之保持不变。

(4) 资源调整。若资源供应发生异常时，应进行资源调整。资源供应发生异常是指供应满足不了需要，例如，资源强度降低或中断，影响计划工期的实现。资源调整的前提是保证工期不变或使工期更加合理。资源调整的方法是进行资源优化，提高资源利用率。

在软件项目中，必须处理好进度与质量之间的关系。在软件开发实践中，常常会遇到这样的事情，当任务未能按计划完成时，只好设法加快进度赶上去。但事实告诉我们，在进度压力下赶任务，其成果往往是以牺牲产品的质量为代价的。因此，当某一开发项目的进度有可能延期时，应该分析延期原因，加以补救；不应该盲目地投入新的人员或推迟预定完成日期。Brooks 曾指出：为延期的软件项目增加人员将可能使其进度更慢。

8.4 预算与成本管理

项目成本是指为完成项目目标而付出的费用和耗费的资源。影响项目成本的因素非常多，而且变化大。在这些因素中，质量、进度和范围对项目成本的影响不但非常突出，而且关联性很强。价格和管理水平对于项目成本也具有重要的影响。

项目成本管理是在整个项目的实施过程中，为确保项目在批准的预算条件下尽可能保质按期完成，而对所需的各个过程进行管理与控制。项目成本管理包括成本估算、成本预算和成本

控制三个过程。成本估算是对完成项目所需成本的估计和计划，是项目计划中的一个重要的、关键的、敏感的部分；成本预算是把估算的总成本分配到项目的各个工作包，建立成本基准计划以衡量项目绩效；成本控制保证各项工作在各自的预算范围内进行。

8.4.1 成本估算

成本估算是对项目投入的各种资源的成本进行估算，并编制费用估算书。要进行项目成本的估算，需要大量的数据资料，这些资料包括资源的类型和数量、每种资源的单价、每项资源占有的时间。因此，成本估算和活动资源估算往往是结合在一起的。

成本估算主要靠分解和类推的手段进行，基本估算方法分为三类：自顶向下的估算、自底向上的估算和差别估算法。

自顶向下估算的主要思想是从项目的整体出发，进行类推。即估算人员根据以前已完成项目所消耗的总成本（或总工作量），来推算将要开发的软件的总成本（或总工作量），然后按比例将它分配到各开发任务单元中去。

自底向上估算的主要思想是把系统进行细分，直到每一个子任务都已经明确所需要的开发工作量，然后把它们加起来，得到系统开发的总工作量。这是一种常见的估算方法。

差别估算法综合了自顶向下估算和自底向上估算的优点，其主要思想是把项目与过去已完成的项目进行类比，从其开发的各个子任务中区分出类似的部分和不同的部分。类似的部分按实际量进行计算，不同的部分则采用相应的方法（例如，德尔菲法等）进行估算。这种方法的优点是可以提高估算的准确程度，缺点是不容易明确“类似”的界限。

8.4.2 成本预算

成本预算是进行项目成本控制的基础，是将估算的成本分配到项目的各项具体工作上，以确定项目各项工作和活动的成本定额，制订项目成本的控制标准，规定项目意外成本的划分与使用规则。成本预算的基本步骤如下：

（1）分摊项目总成本到 WBS 的各个工作包中，为每个工作包建立总预算成本。在将所有工作包的预算成本相加时，结果不能超过项目的总预算成本。

（2）将每个工作包分配得到的成本再二次分配到工作包所包含的各项活动上。

（3）确定各项成本预算支出的时间计划，以及每个时间点对应的累积预算成本，编制项目成本预算计划。

1. 直接成本与间接成本

在进行成本预算时，除了要考虑项目的直接成本，还要考虑其间接成本和对成本有影响的其他因素，可能包括以下几项：

（1）非直接成本。包括租金、保险和其他管理费用。

（2）隐没成本。隐没成本（沉没成本）是已经付出且不可收回的成本。

（3）学习曲线。在信息系统项目中，如果采用了项目组成员未使用过的技术和方法，那么，在使用这些技术和方法的初期，项目组成员有一个学习的过程，许多时间和劳动投入到尝试和

试验中。这些尝试和试验会增加项目的成本。同样,对于项目组从未从事的项目要比对原有项目的升级的成本高得多,也是由于项目组必须学习新行业的术语、原理和流程。

(4) 项目完成的时限。一般来说,项目需要完成的时限越短,那么成本就越高,压缩信息系统的交付日期,不仅要支付项目组成员的加班费用,而且如果过于压缩进度,项目组可能在设计和测试上就会减少投入,项目的风险会提高。

(5) 质量要求。显然,成本估算要根据产品质量要求的不同而不同。

(6) 保留。保留是为风险和未预料的情况而准备的预留成本。

2. 管理储备

管理储备是为范围和成本的潜在变化而预留的预算,它们是未知的,项目经理在使用之前必须得到批准。管理储备不是项目成本基线的一部分,但包含在项目的预算中。它们未被作为预算进行分配,因此,不是挣值分析的一部分。

3. 零基准预算

零基准预算是指在项目预算中,并不以过去相似的项目成本作为成本预算的基准,然后根据项目之间的规模、性质、质量要求、工期要求等不同,对基准进行调节来对新的项目进行成本预算。而是项目以零作为基准,估计所有的工作任务的成本。

8.4.3 成本控制

项目成本控制是按照事先确定的成本基准计划,通过运用多种恰当的方法,对项目实施过程中所消耗费用的使用情况进行管理和控制,以确保项目的实际成本限制在项目成本预算范围内。成本控制必须识别可能引起项目成本基准计划发生变动的因素,并对这些因素施加影响,以保证该变化朝着有利的方向发展。监督费用实施情况,发现实际费用和成本计划的偏差,并找出偏差的原因,阻止不正确、不合理和未经批准的费用变更。

在项目成本控制中,主要采用挣值分析方法。挣值分析是一种进度和成本测量技术,用来估计和确定变更的程度和范围,因此,也称为偏差分析法。挣值分析通过测量和计算已完成工作的预算费用、已完成工作的实际费用和计划工作的预算费用,得到有关计划实施的进度和费用偏差,达到判断项目预算和进度计划执行情况的目的。

8.5 配置管理

配置是在技术文档中明确说明并最终组成软件产品的功能或物理属性,包括即将受控的所有产品特性,其内容及相关文档、软件版本、变更文档、软件运行的支持数据,以及其他一切保证软件一致性的组成要素。

软件配置管理(Software Configuration Management, SCM)是指通过执行版本控制、变更控制的规程,以及使用合适的配置管理工具,来保证所有配置项的完整性和可跟踪性。配置管理是对工作成果的一种有效保护。

8.5.1 配置管理概述

根据国家标准《信息技术 软件工程术语》（GB/T 11457—2006），配置管理是标识和确定系统中配置项的过程，是在系统整个生存期内控制这些配置项的投放和更动，记录并报告配置的状态和变动要求，验证配置项的完整性和正确性，并对下列工作进行技术和行动指导与监督的一套规范。配置管理包括以下内容：

- （1）对配置项的功能特性和物理特性进行标识和文件编制工作。
- （2）控制这些特性的变动情况。
- （3）记录并报告这些变动进行的处理和实现的状态。

配置管理的目的在于运用配置标识、配置控制、配置状态和配置审计，建立和维护工作产品的完整性。CMMI 把配置管理分为九大部分，分别是制订配置管理计划、识别配置项、建立配置管理系统、创建或发行基线、跟踪变更、控制变更、建立配置管理记录、执行配置审核、版本控制。而国家标准《系统与软件工程 软件生存周期过程》（GB/T 8566—2022）所规定的软件配置管理过程的活动有编制配置管理计划、配置标识、变更管理和配置控制、配置状态说明、配置审核、版本控制和发行管理。

1. 配置管理过程

配置管理过程包括以下内容：

（1）编制软件配置管理计划。在项目启动时，项目经理首先要制订整个项目的开发计划，它是整个软件开发工作的基础。配置管理计划是项目开发计划的一部分。

（2）配置标识。确定哪些内容应该进入配置管理，形成配置项，并确定配置项如何命名，用哪些信息来描述配置项。配置标识是配置管理的基础性工作，是进行配置管理的前提。

（3）变更管理和配置控制。配置管理最重要的任务就是对（配置项的）变更加以控制和管理，其目的是对于复杂而无形的软件，防止在多次变更下失控，出现混乱。

（4）配置状态说明。配置状态说明也称为配置状态报告，其任务是有效地记录、报告管理配置所需要的信息，目的是及时、准确地给出配置项的当前状况，供相关人员了解，以加强配置管理工作。

（5）配置审核。配置审核的任务是验证配置项对配置标识的一致性。软件开发的实践表明，尽管对配置项做了标识，实现了变更控制和版本控制，但如果不做检查或验证，仍然会出现混乱。配置审核的实施是为了确保软件配置管理的有效性，体现配置管理的最根本要求，不允许出现任何混乱现象。

（6）版本控制和发行管理。在配置管理中，版本包括配置项的版本和配置的版本，这两种版本的标识应该各有特点，配置项的版本应该体现出其版本的继承关系，它主要是在开发人员内部进行区分。另外，还需要对重要的版本做一些标识，例如，对纳入基线的配置项版本就应该做一个标识。

2. 配置管理计划

根据国家标准《计算机软件文档编制规范》（GB/T 8567—2006）的规定，配置管理计划应

该包括以下几部分内容：

(1) 引言。包括配置管理计划的标识、系统概述、文档概述、组织和职责、配置管理活动所需的各种资源等。

(2) 引用文件。列出配置管理计划中引用的所有文档的编号、标题、修订版本和日期。

(3) 管理。描述在各阶段负责 SCM 的机构和任务，以及要进行的评审和检查工作，指出各阶段的产品应存放在哪一类配置库中；指出各机构的职责和它们之间的关系，描述相关的接口控制；规定实现 SCM 计划的主要里程碑；指明 SCM 适用的标准和规范，以及这些标准和规范要实现的程度。

(4) SCM 活动。描述配置标识、配置控制、配置状态记录与报告、配置检查与评审等 4 个方面的 SCM 活动。

(5) 工具、技术和方法。指明为支持特定项目的 SCM 所使用的软件工具、技术和方法，以及它们的目的，并在开发人员所有权的范围内描述其用法。

(6) 对供货单位的控制。规定对供货单位进行控制的规程，从而使从软件销售单位购买的、其他开发单位开发的或从开发单位现存软件库中选用的软件能满足规定的 SCM 需求。

(7) 记录的收集、维护和保存。规定要保存的 SCM 文档，以及用于汇总、保护和维护工程文档的方法和设施（其中包括要使用的后备设施），并说明要保存的期限。

(8) 配置项和基线。根据企业的有关规范，对不同类型的配置项建立命名规则，列出识别到的所有配置项和所属的配置基线，并明确配置项的标识、作者（或负责人）和配置时间。描述配置项和基线变更、发布的流程，以及相应的批准权限。

(9) 备份。说明配置库和配置管理库的备份方式、频率和责任人。

(10) 日程表。列出 SCM 活动的日程表，并确保配置管理活动的日程表与项目开发计划、质量管理计划保持一致。

(11) 注解。包含有助于理解配置管理计划的一般信息，例如，背景信息、词汇表、原理等。这一部分应包含理解配置管理计划所需要的术语和定义，所有缩略语和它们在配置管理计划中的含义的字母序列列表。

(12) 附录。提供那些为便于维护配置管理计划而单独编排的信息（例如，图表、分类数据等）。为便于处理，附录可以单独装订成册，按字母顺序编排。

8.5.2 配置标识

软件开发中的文档和软件在其开发、运行、维护的过程中会得到许多阶段性的成果，在开发和运行过程中还需要用到多种工具软件或配置。所有这些信息项都需要得到妥善的管理，决不能出现混乱，以便在提出某些特定的要求时，将它们进行约定的组合来满足使用的目的。这些信息项是配置管理的对象，称为配置项。

每个配置项的主要属性有名称、标识符、状态、版本、作者、日期等。配置项是一个独立存在的信息项，可以把它看成一个元素，单独的一个元素发挥不了什么作用，但随着工作的进展，出于不同的要求，需要将这些元素进行不同的组合，这个组合称为配置。配置是一个信息系统产品在生存期各个阶段的不同形式（记录特定信息的不同媒体）和不同版本的程序、文档

及相关数据的集合，或者说是配置项的集合，它具有完整的意义。

1. 确定配置项

软件项目中形成的技术性文档和管理性文档，除一些临时性的文档外，一般都应该进行配置管理。一般来说，判定一个文档是否进行配置管理的标准应该是此文档是否有多个人需要使用，这些文档往往在开发的过程中不断地修正和扩展，要保证每个使用者都使用同一版本的文档，就必须将这些文档纳入配置管理，成为受控的配置项。

(1) 识别配置项。可能成为配置项组成部分的主要工作产品有过程描述、需求、设计、测试计划和规程、测试结果、代码/模块、工具、接口描述等。在软件工程方面，Roger S. Pressman 认为至少以下所列的文档应该成为配置项：系统规格说明书、项目计划、需求规格说明书、用户手册、设计规格说明、源代码、测试规格说明、操作和安装手册、可执行程序、数据库描述、联机用户手册、维护文档、软件工程标准和规程。

(2) 配置项命名。确定了配置项后，还需要对配置项进行合理、科学的命名。配置项的命名绝不能随意为之，必须满足唯一性和可追溯性。一个典型的实例是采用层次式的命名规则来反映树状结构，树状结构上节点之间存在着层次的继承关系。

(3) 配置项的描述。由于配置项除了名称外还有一些其他属性和与其他配置项的关系，因此，它可以采用描述对象的方式来进行描述。每个配置项用一组特征信息（名字、描述、一组资源、实现）唯一地标识。配置项间的关系有整体和部分的的关系及层次关系，也有关联关系。配置项间的关系可以用 MIL (Module Interconnection Language, 模块内连接语言) 表示，MIL 描述的是配置项间的相互依赖关系，可自动构造系统的任何版本。

2. 基线

基线 (Baseline) 是项目生存期各开发阶段末尾的特定点，也称为里程碑 (Milestone)，在这些特定点上，阶段工作已结束，并且已经形成了正式的（通过了正式的技术评审）阶段产品。

建立基线的概念是为了把各开发阶段的工作划分得更加明确，使得本来连续开展的开发工作在这些点上被分割开，从而更加有利于检验和肯定阶段工作的成果，同时有利于进行变更控制。有了基线的规定，就可以禁止跨越里程碑去修改另一开发阶段的工作成果，并且认为建立了里程碑，有些完成的阶段成果已被冻结。

如果把软件看作系统的一个组成部分，以下三种基线是最受人们关注的，分别是功能基线、分配基线和产品基线。

(1) 功能基线。功能基线指在系统分析与软件定义阶段结束时，经过正式评审和批准的系统设计规格说明书中对待开发系统的规格说明；或是指经过项目业主和承建单位双方签字同意的协议书或合同中所规定的对待开发软件系统的规格说明；或是由下级申请经上级同意或直接由上级下达的项目任务书中所规定的对待开发软件系统的规格说明。功能基线是最初批准的功能配置标志。

(2) 指派基线 (分配基线)。指派基线指在软件需求分析阶段结束时，经过正式评审和批准的 SRS。指派基线是最初批准的指派配置标志。

(3) 产品基线。产品基线指在软件组装与系统测试阶段结束时, 经过正式评审和批准的有所开发的软件产品的全部配置项的规格说明。产品基线是最初批准的产品配置标志。

另外, 交付给项目组所在企业的外部客户的基线一般称为发行基线, 企业内部使用的基线称为构造基线。释放(Release)是指在软件生存周期的各个阶段结束时, 由该阶段向下一阶段提交该阶段产品的过程; 它也指将系统测试阶段结束时所获得的最终产品向用户提交的过程。后面这个过程也称为交付(Delivery)。

提出基线的概念本来是为了更好地实现变更控制, 但如果把每个基线都当成一个整体来看待会造成麻烦。因为一个变更很可能只涉及基线的很小一部分。例如, 假定某个大型软件中的一个模块修改了, 如果将这一变更当作整个软件产品基线的变更, 就很不方便。

3. 建立配置管理系统

在配置管理中, 要建立并维护配置管理系统和变更管理系统。建立配置管理系统的主要步骤如下:

(1) 建立适用于多控制等级配置管理的管理机制。软件生存周期不同时间所需的控制等级不同, 不同的系统类型所需的控制等级也不同。

(2) 存储和检索配置项。

(3) 共享和转换配置项。

(4) 存储和复原配置项的归档版本。

(5) 存储、更新和检索配置管理记录。

(6) 创建配置管理报告。

(7) 保护配置管理系统的内容。配置管理系统的主要功能有文档的备份与恢复、文档的建档、从配置管理的差错状态下复原。

(8) 权限分配。配置管理员为每个项目成员分配对配置库的操作权限。配置管理员的权限最高, 一般项目成员可拥有添加、检入/检出(Check In/Check Out)、下载的权限, 但是不能有删除的权限。

4. 配置库

配置库也称为配置项库, 是用来存放配置项的工具。配置库记录与配置相关的所有信息, 其中存放的受控的配置项是很重要的内容, 利用配置库中的信息可评价变更的后果, 这对变更控制有着重要的意义。配置库有三类:

(1) 开发库。存放开发过程中需要保留的各种信息, 供开发人员个人专用。开发库中的信息可能有较为频繁的修改, 只要使用者认为有必要, 无须对其做任何限制。因为这通常不会影响项目的其他部分。开发库对应配置管理系统中的动态系统(开发者系统、开发系统、工作空间)。

(2) 受控库。在开发的某个阶段工作结束时, 将工作产品存入或将有关的信息存入。存入的信息包括计算机可读的和人工可读的文档资料。应该对受控库内的信息的读写和修改加以控制。受控库也称为主库, 对应配置管理系统中的主系统(受控系统)。

(3) 产品库。在开发的产品完成系统测试之后, 作为最终产品存入产品库内, 等待交付用

户或现场安装。产品库内的信息也应加以控制。产品库也称为备份库，对应配置管理系统中的静态系统。

8.6 质量管理

国家标准《系统与软件工程 系统与软件质量要求和评价（SQuaRE）第10部分：系统与软件质量模型》（GB/T 25000.10—2016）中对软件质量的定义为“在规定条件下使用时，软件产品满足明确或隐含要求的能力”。

根据《质量管理体系 基础和术语》（GB/T 19000—2016）的定义，质量管理是指确立质量方针及实施质量方针的全部职能及工作内容，并对其工作效果进行评价和改进的一系列工作。ISO 9000 系列标准是现代质量管理的结晶，它实际上是由计划、控制和文档工作三个部分组成循环的体系。ISO 9000 标准是以质量管理中的8项原则为基础的，分别是以顾客为关注焦点、领导作用、全员参与、过程方法、管理的系统方法、持续改进、以事实为基础进行决策、与供方互利的关系。

8.6.1 软件质量模型

软件或系统的质量是指该软件或系统满足其不同利益相关方所明确的或隐含的要求的程度，并由此提供相应的价值。在ISO/IEC 25000 系列标准中，把产品质量分为一些特性，并在某些情况下进一步分解为子特性（某些子特性又被分解为子子特性），通过这种层次化的分解提供了便利的产品质量分级。而一个系统可测量的、与质量相关的属性被称为质量测度相关的质量属性，从而形成了质量特性、质量子特性和质量属性的层级关系。质量模型是定义的特征集以及它们之间的关系集，为规约质量需求以及评价质量提供了一个框架。

GB/T 25000.10—2016 标准中定义了使用质量模型和产品质量模型。由这两个模型所定义的特性，与所有的软件产品和计算机系统有关。这些特性和子特性为规定、测量和评价系统与软件产品质量提供了一致的术语。它们还提供了一组质量特性，依此可比较所陈述的质量需求的完备程度。

1. 使用质量模型

系统的使用质量描述了产品（系统或软件产品）对利益相关方造成的影响。它是由软件、硬件、运行环境的质量，以及用户、任务和社会环境的特性所决定的。所有这些要素均有利于系统的使用质量。

GB/T 25000.10—2016 标准中的使用质量模型如图8-4所示。该模型由5个特性组成，每个特性又可进一步细分为一些子特性，这些特性关系到产品在特定的使用周境中使用时的交互结果。这一系统模型可以应用于整个人机系统，既包括使用中的计算机系统，也包括使用中的软件产品。

使用质量模型的5个特性是：有效性、效率、满意度、抗风险和周境覆盖。每个特性都可以被赋予到利益相关方的不同活动中，例如操作人员的交互或开发人员的维护。

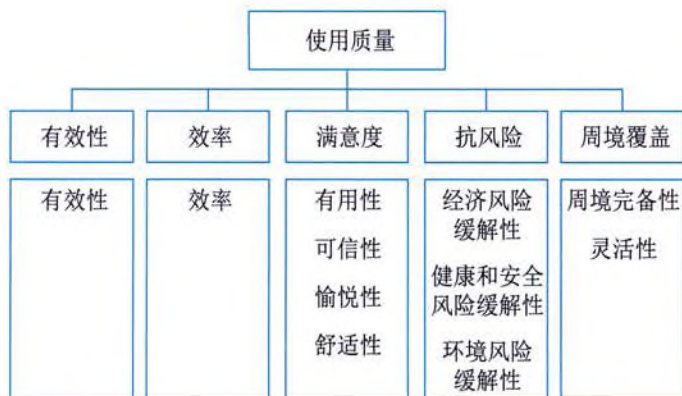


图 8-4 使用质量模型

2. 产品质量模型

产品质量模型如图 8-5 所示，它将系统 / 软件产品质量属性划分为 8 个特性：功能性、性能效率、兼容性、易用性、可靠性、信息安全性、维护性和可移植性。每个特性由一组相关子特性组成。这些特性关系到软件的静态性质和计算机系统的动态性质。这一模型既可以应用于计算机系统，也可以应用于软件产品。



图 8-5 产品质量模型

3. McCall 质量模型

McCall 质量模型从软件运行、软件修改和软件转移三个方面来考查软件的质量，其体系如图 8-6 所示。

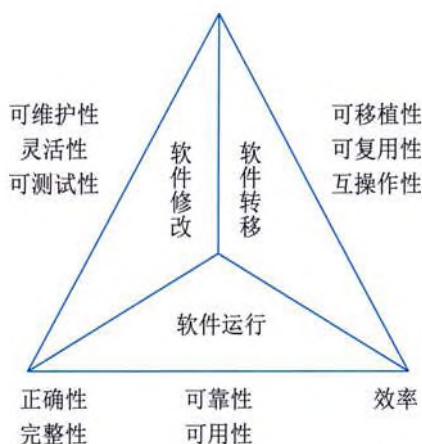


图 8-6 McCall 质量模型

4. 质量特性度量

软件质量特性度量有两类：预测型和验收型。预测度量是利用定量或定性的方法估算软件质量的评价值，以得到软件质量的比较精确的估算值。验收度量是在软件开发各阶段的检查点对软件的要求质量进行确认性检查的具体评价值，它是对开发过程中的预测进行评价。

预测度量有两种。第一种是尺度度量，这是一种定量度量。它适用于一些能够直接度量的特性，例如，出错率定义为“（错误数/kLOC）/单位时间”。第二种是二元度量，这是一种定性度量。它适用于一些只能间接度量的特性，例如可使用性、灵活性等。

要注意的是，在软件质量的所有子特性中，有些是正相关的，有些是负相关的。例如，可靠性和可用性就是正相关的，它们之间可以互相促进；而安全性和效率就是负相关的，它们之间是一对矛盾体。因此，在系统设计过程中应根据具体情况对各种特性进行折中，以便得到在总体上使用户满意的质量标准。

5. 相关概念

在质量管理部分，读者还需要理解与软件质量相关的三个概念：

- （1）验证：指通过检查和提供客观的证据来证实规定需求已经得到满足。
- （2）确认：指通过检查和提供客观的证据来证实针对某一特定预期用途的需求已经得到满足。
- （3）测量：指一组操作，其目的是确定某个测度的值。

8.6.2 质量管理计划

现代质量管理的一项基本准则是“质量是计划出来的，而不是检查出来的”，这是在项目质量管理工作中必须牢牢把握的。编制一份清晰的质量管理计划是实施项目质量管理的第一步，而一个清晰的质量管理计划首先需要明确以下两点：一是明确将采用的质量标准；二是明确质量目标。

在该过程中，质量政策和质量标准往往是编制质量管理计划的约束条件，并来自于项目组织之外。例如，质量政策可能来自于项目执行组织高层的战略规划，质量标准可能来自于强制

性的行业标准或国家标准等。但并不是说项目团队对于上述两方面无能为力，其实，项目团队本身也可以在一定程度上对它们施加自己的影响。例如，通过功能分析和价值分析来对质量政策和质量标准进行权衡。另外，项目质量管理计划的编制必须结合信息系统项目的具体特征和企业自身的实际情况。

8.6.3 质量保证与质量控制

项目质量管理的主要活动是质量保证与质量控制。质量保证是指定期评估项目总体绩效，建立项目能达到相关质量标准的信心。质量保证对项目的最终结果负责，而且还要对整个项目过程承担质量责任。质量控制是指监测项目的总体结果，判断它们是否符合相关质量标准，并找出消除不合格绩效的方法。

1. 质量保证

软件质量保证（Software Quality Assurance, SQA）是指为保证软件系统或软件产品充分满足用户要求的质量而进行的有计划、有组织的活动，这些活动贯穿软件生产的各个阶段，即整个生命周期。SQA 由各项任务构成，这些任务的参与者有两种人：软件开发人员和质量保证人员。前者负责技术工作，后者负责质量保证的计划、监督、记录、分析及报告工作。软件开发人员通过采用可靠的技术、方法和措施，进行正式的技术评审，执行软件测试来保证软件产品的质量。质量保证人员则辅助软件开发人员得到高质量的最终产品。

2. 质量控制

质量控制是为使产品或服务达到质量要求而采取的技术措施和管理措施方面的活动，也包括判断如何能够去除造成不合格结果的根源。质量控制应贯穿项目始终。

项目结果既包括产品结果（例如，可交付成果），也包括项目管理结果（例如，成本与进度绩效）。因此，项目的质量控制主要从项目产品 / 服务的质量控制和项目管理过程的质量控制两个方面进行，其中，项目管理过程的质量控制是通过项目审计来进行的，项目审计是将管理过程的任务与成功实践的标准进行比较所做的详细检查。

3. 质量保证与质量控制的关系

质量保证一般是每隔一定时间（例如，每个阶段末）进行的，主要通过系统的质量审计和过程分析来保证项目的质量。质量控制是实时监控项目的具体结果，以判断它们是否符合相关质量标准，制定有效方案，以消除产生质量问题的原因。一定时间内质量控制的结果也是质量保证的质量审计对象。质量保证的成果又可以指导下一阶段的质量工作，包括质量控制和质量改进。

8.7 人力资源管理

项目人力资源管理是指通过不断地获得人力资源，把得到的人力资源整合到项目中并融为一体，保持和激励团队成员对项目的忠诚和积极性，控制团队成员的工作绩效并做出相应的调整，尽量发挥团队成员的潜能，以支持项目目标实现的活动、职能、责任和过程。项目人力资

源管理的主要过程包括编制人力资源计划、组建项目团队、项目团队建设、管理项目团队和沟通管理。

8.7.1 编制人力资源计划

人力资源计划涉及决定、记录和分配项目角色、职责及报告关系的过程，描述项目的角色和职责的工具主要有层次结构图、矩阵图和文本格式的角色描述。文本格式用来详细描述团队成员的职责，提供的信息主要有职责、权力、能力和资格。

1. 组织结构图

在生成项目组织结构图之前，高层管理者和项目经理必须明白什么类型的人是真正保证项目的关键人物，他们需要什么样的技能。例如，如果需要找一些优秀的系统分析师，则人力资源计划就需要反映这个需求；如果项目成功的关键是需要一流的项目经理和被人尊敬的团队领导，人力资源计划也要重点描述。

组织结构图通常描述为一个层次结构。例如，图 8-7 就是一个项目的组织结构图。

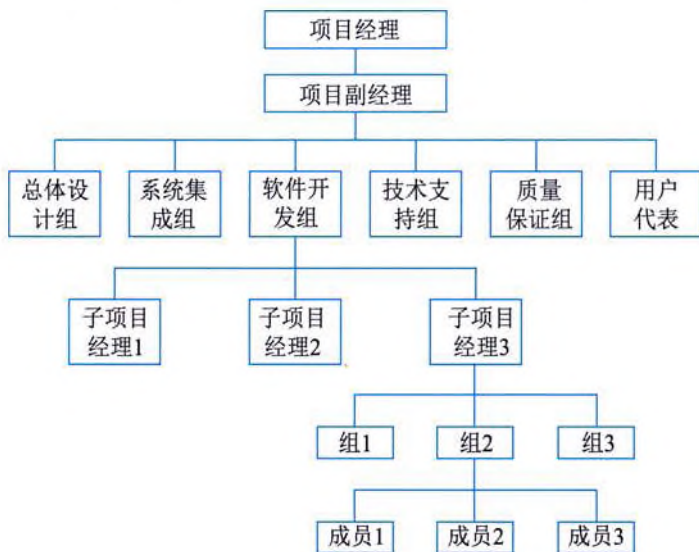


图 8-7 组织结构图示例

2. 责任分配矩阵

项目工作一旦分解成可管理的元素，项目经理就可以给组织单元分配任务了，主要是基于“适合优先”的原则来分配任务，这个过程可以用组织分解结构（Organizational Breakdown Structure, OBS）来进行概念化的描述。OBS 类似于 WBS，是一种用于表示组织单元负责哪些工作内容的特定的组织图形。它可以先借用一个通用的组织图形，然后针对组织或分包商中特定部门的单元进行逐步细分。

在制作完 OBS 之后，就可以开发责任分配矩阵（Responsibility Assignment Matrix, RAM）了。RAM 为项目工作（用 WBS 表示）和负责完成工作的人（用 OBS 表示）建立一个映射关

系。除了将 RAM 用于具体的工作任务分配之外, RAM 还可以用于定义角色和职责间的关系。此时, RAM 包括项目干系人, 表 8-3 给出一个例子, 表明不同类型的项目干系人在项目过程中的责任, 是负责人 (A) 还是参与者 (P), 是为项目过程提供输入 (I), 还是评审 (R) 或签字确认者 (S)。这个看似简单的表格为项目经理提供了一种有效地管理项目重要干系人和角色的工具。

表 8-3 体现项目干系人角色的 RAM

阶 段	项目干系人				
	A	B	C	D	E
单元测试	S	A	I	I	R
集成测试	S	P	A	I	R
系统测试	S	P	A	I	R
验收测试	S	P	I	A	R

另外一种形式的 RAM 中的符号标记为 RACI, 其中 R 代表对任务负责任, A 代表负责执行任务, C 代表提供信息辅助执行任务, I 代表拥有既定特权、应及时得到通知。

8.7.2 组建项目团队

组建项目团队的主要任务是根据项目开发计划, 获取完成项目工作所需的人力资源。能否组建一个满足项目需要的团队, 是项目能否获得成功的基本条件和关键所在。项目经理应从各种来源物色团队成员, 将符合要求的人员纳入项目团队, 将计划编制阶段确定的角色连同责任分配给各个成员, 并明确他们之间的配合、汇报和从属关系。项目团队成员既可来自企业内部, 也可来自企业外部。

项目团队的组建是一个动态的过程, 即随着项目的进展, 对人员的需求是动态变化的。例如, 项目前期需要更多的系统分析师的参与, 而项目后期却需要更多的程序员的参与。项目经理必须能够监控这种变化, 在人员技能与项目需求不一致的情况下, 及时与企业高层、人力资源经理等进行沟通, 以保证项目对人员的动态需求。

8.7.3 项目团队建设

项目团队建设的主要任务是, 提高项目团队成员的个人技能, 以提高他们完成项目的活动的能力, 与此同时, 降低成本、缩短工期、改进质量并提高绩效; 提高项目团队成员之间的信任感和凝聚力, 以提高士气, 降低冲突, 促进团队合作; 创建动态的、团结合作的团队文化, 以促进个人与团队的生产率、团队精神和团队协作, 鼓励团队成员之间交叉培训和切磋, 以共享经验和知识。

根据团队规模、团队成员的素质、团队组建时间的长短等方面的不同, 可以采取不同的措施来建设项目团队, 包括培训、集中办公、团队建设活动、恰当的奖励与表彰措施等。

项目团队从开始到终止, 是一个不断成长和变化的过程, 这个发展过程可以描述为 4 个时期: 形成期、震荡期、正规期、表现期。几乎所有的项目都经历过大家被召集到一起的形成期,

这是一个短暂的时期。很快进入震荡期，这时成员之间互相还不了解，时常感到困惑，有时甚至会产生敌对心理。接下来在强有力的领导下，团队的工作方式在正规期得以统一。随后团队进入表现期，以最大成效开展工作，直至项目结束，项目团队解散。项目团队各阶段的团队精神、工作绩效、调控力度的示意图如图 8-8 所示。

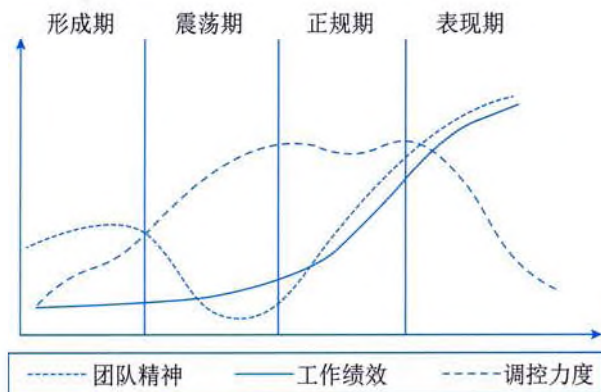


图 8-8 项目团队各阶段示意图

8.7.4 管理项目团队

管理项目团队是指跟踪个人和团队的绩效，提供反馈，解决问题和协调变更，以提高项目的绩效。项目经理必须观察团队的行为、管理冲突、解决问题和评估团队成员的绩效。

一旦项目成员被分配到项目中，项目经理有两种方法来最有效地使用项目团队中的成员，即资源负荷和资源平衡。

资源负荷是指在特定的时间内，现有的进度计划所需要的各种资源的数量。如果在特定的时间内分配给某项工作的资源超出了项目的可用资源，则称为资源超负荷。资源超负荷本身就是一种资源冲突的现象，为了消除超负荷，项目经理可以修改进度表，尽量使资源得到充分的利用或者充分利用项目活动的时差，这种方法就叫做资源平衡。

资源平衡是一种延迟项目任务来解决资源冲突问题的方法，是一种网络分析法，它将以资源管理因素为主进行项目进度决策。资源平衡的主要目的是更加合理地分配使用的资源，使项目的资源达到最有效的利用，资源平衡的时候，资源的利用也就达到了最佳的状态。

8.7.5 沟通管理

项目沟通管理的目标是及时而适当地创建、收集、发送、储存和处理项目的信息。项目经理的大部分时间花在沟通上，包括内部沟通和外部沟通。作为技术骨干的系统分析师，对内需要与项目团队和睦相处，协助项目经理做好技术管理工作；对外需要与用户进行良好的沟通，以便正确而快捷地获取用户需求，做好系统分析与设计工作。

沟通有正式沟通和非正式沟通之分。正式沟通是通过项目组织明文规定的渠道进行信息传递和交流的方式。它的优点是沟通效果好，有较强的约束力，缺点是沟通速度慢。非正式沟通是指在正式沟通渠道之外进行的信息传递和交流。这种沟通的优点是沟通方便，沟通速度快，

且能提供一些正式沟通中难以获得的信息，缺点是容易失真。

会议是项目沟通的一种重要形式。一个成功的会议能成为鼓励项目团队建立和加强对项目的期望、任务、关系和责任的工具。失败的会议会对一个项目产生负面的影响。

8.8 风险管理

风险管理就是要对项目风险进行认真的分析和科学的管理，这样能够避开不利条件、少受损失、取得预期的结果并实现项目目标，能够争取避免风险发生或尽量减小风险发生后的影响。但是，完全避开或消除风险，或者只享受权益而不承担风险是不可能的。

8.8.1 风险管理的概念

项目风险管理就是项目管理人员通过风险识别、风险估计和评价，并以此为基础合理地使用多种管理方法、技术和手段，对项目活动涉及的风险实行有效的控制，采取主动行动、创造条件、可靠地实现项目的总体目标。

风险表达了一种概率，具有偶发性。对于项目中的风险可以简单地理解为项目中的不确定因素。从广义的角度说，不确定因素一旦确定了，既可能对当前情况产生积极的影响，也可能产生消极的影响。也就是说，风险发生后既可能给项目带来问题，也可能给项目带来机会。

风险具有两个基本属性：随机性和相对性。随机性是指风险事件的发生及其后果都具有偶然性；相对性是指风险总是相对项目活动主体而言的，同样的风险对于不同的主体有不同的影响。

8.8.2 风险管理的过程

项目需要以有限的成本，在有限的时间内达到目标，而风险会影响这一点。风险成本是指风险事件造成的损失或减少的收益，以及为防止发生风险事件采取预防措施而支付的费用。风险成本可以分为有形成本、无形成本，以及预防与控制风险的费用。有形成本包括直接损失和间接损失，直接损失是指财产损毁和人员伤亡的价值，间接损失是指直接损失以外的其他损失；无形成本是指由于风险所具有的不确定性而使项目主体在风险事件发生之前或发生之后付出的代价，主要表现在风险损失或减少了机会、风险阻碍了生产率的提高、风险造成资源分配不当。

风险管理的目的就是最小化风险对项目目标的负面影响，抓住风险带来的机会，增加项目干系人的收益。项目风险管理的基本过程包括风险管理计划编制、风险识别、风险定性分析、风险定量分析、风险应对计划编制和风险监控。

1. 风险管理计划编制

风险管理计划描述的是如何安排与实施项目风险管理，它是项目开发计划的从属计划。风险管理计划主要包括角色与职责、预算、风险类别、风险概率和影响的定义、汇报格式、风险

跟踪等内容。

很多项目除了编制风险管理计划之外，还有应急计划和应急储备。应急计划是指当一项可能的风险事件实际发生时项目团队将采取的预先确定的措施；应急储备是指根据项目发起人的规定，如果项目范围或者质量发生变更，这一部分资金可以减少成本或进度风险。

2. 风险识别

风险识别就是采用系统化的方法，识别出项目中已知的和可预测的风险。风险识别是一项反复的过程，项目团队应该参与该过程，以便形成针对风险的应对措施，并保持一种责任感。

风险识别包括确定风险的来源、风险产生的条件，描述风险特征和确定哪些风险事件有可能影响整个项目。风险识别应当在项目的生命周期自始至终定期进行。风险识别可分为三步进行：收集资料、估计项目风险形势、根据直接或间接的症状将潜在的风险识别出来。

3. 风险定性分析

风险定性分析是指对已识别风险进行优先级排序，以便采取进一步措施。主要包括以下内容：

（1）风险可能性与影响分析。可能性评估需要根据风险管理计划中的定义，确定每一个风险发生的可能性，并记录下来。除了风险发生的可能性，还应当分析风险对项目的影响。风险影响分析应当全面，需要包括对时间、成本、范围等各方面的影响。

（2）确定风险优先级。在确定了风险的可能性和影响后，需要进一步确定风险的优先级。风险优先级是一个综合的指标，优先级的高低反映了风险对项目的综合影响，也就是说，高优先级的风险最可能对项目造成严重的影响。风险优先级的概念与风险可能性和影响既有联系又不完全相同。

（3）确定风险类型。在进行风险定性分析的时候需要确定风险的类型。

4. 风险定量分析

风险定量分析是在不确定的情况下进行决策的一种量化方法，该过程主要采用灵敏度分析、期望货币价值分析、决策树分析、蒙特卡罗模拟等技术。下面，只简单介绍蒙特卡罗模拟。

蒙特卡罗（MonteCarlo）方法作为一种统计模拟方法在各行业被广泛运用。蒙特卡罗方法在定量分析中的运用较为复杂，牵涉复杂的数理概率模型。它将一个多元函数的取值范围问题分解为若干个主要参数的概率问题，然后用统计方法进行处理，得到该多元函数的综合概率，在此基础上分析该多元函数的取值范围可能性。蒙特卡罗方法的关键是找一组随机数作为统计模拟之用，这一方法的精度在于随机数的均匀性与独立性。就运用于风险分析而言，蒙特卡罗方法主要通过分析各种不确定因素，灵活地模拟真实情况下的某个系统中的各主要因素变化对风险结果的影响。由于计算过程极其繁复，蒙特卡罗方法不适合简单（单变量）模型，而对于复杂（有多种不确定因素）模型则是一种很好的方法。其具体步骤包括选取变量、分析各变量的概率分布、选取各变量的样本、模拟价值结果、分析结果。

5. 风险应对计划编制

制订风险应对计划时有多种不同的策略，对于相同的风险，采用不同的应对策略会有不同

的应对方法。通常可以把风险应对策略分为两种类型：防范策略和响应策略。防范策略指的是在风险发生前，项目团队会采取一定的措施对风险进行防范；而响应策略则是在风险发生后采取的相应措施，以降低风险带来的损失。

消极的风险（负面风险，威胁）防范策略是最常用的策略，其目的是降低风险发生的概率或减轻风险带来的损失。例如，避免策略、转移策略和减轻策略等。对于正向风险（机会）的应对策略也有三种：开拓、分享和强大（提高）。

风险响应策略与风险防范策略不同，无论风险是否发生，风险防范策略都需要体现在项目计划中，在项目过程中需要有人来执行相应的防范策略。而风险响应策略是事件触发的，直到风险发生后才会被执行，如果始终没有发生该风险，则始终不会被安排到项目活动中。

6. 风险监控

风险监控是指跟踪已识别的风险，监测残余风险和识别新的风险，保证风险计划的执行，并评价这些计划对减轻风险的有效性。

8.9 信息（文档）管理

软件文档是软件产品的重要组成部分，对于开发人员、管理人员以及用户都是十分重要的辅助工件。定义清晰、维护及时的文档能够帮助开发人员理解需求、顺畅沟通；帮助管理人员了解进度、加强管理；帮助用户更好地使用和维护软件。因此，对于系统分析师而言，必须掌握软件文档编制的技能。

8.9.1 软件文档标准

在软件文档方面，国家制定了一些标准，主要有《计算机软件文档编制规范》（GB/T 8567—2006）。

GB/T 8567—2006 主要对软件的开发过程和管理过程应编制的主要文档及其编制的内容、格式规定了基本要求。该标准原则上适用于所有类型的软件产品的开发过程和管理过程。

GB/T 8567—2006 规定了文档过程，包括软件标准的类型（含产品标准和过程标准）、源材料的准备、文档计划、文档开发、评审、文档编制要求、文档编制格式。详细给出了 25 种文档编制的格式，包括可行性分析（研究）报告、软件开发计划、软件测试计划、软件安装计划、软件移交计划、运行概念说明、系统/子系统需求规格说明、接口需求规格说明、系统/子系统（结构设计）说明、接口设计说明、软件需求规格说明、数据需求说明、软件（结构）设计说明、数据库（顶层）设计说明、软件测试说明、软件测试报告、软件配置管理计划、软件质量保证计划、开发进度月报、项目开发总结报告、软件产品规格说明、软件版本说明、软件用户手册、计算机操作手册、计算机编程手册。这 25 种文档可分别适用于计算机软件的管理人员、开发人员、维护人员和用户。该标准给出了 25 种文档的具体内容，使用者可根据实际情况对该标准进行适当剪裁。

GB/T 8567—2006 还规定面向对象的软件应编制以下文档：总体说明文档、用例图文档、类图文档、顺序图文档、协作图（通信图）文档、状态图文档、活动图文档、构件图文档、部

署图文档、包图文档。

8.9.2 数据需求说明

如果项目中有大量的各种数据需要处理，而且这些数据的采集、加工对系统来说有着十分重要的作用，或者是有大量的原始数据需要导入新的系统，那么就有必要对这些数据要求进行分析，并编制数据需求说明（数据要求规格说明书）。数据需求说明的编制目的是向整个开发时期提供关于被处理数据的描述和数据采集要求的技术信息。

根据 GB/T 8567—2006 的规定，数据需求说明应该包括以下几部分内容：

（1）引言。包括数据需求说明的标识（包括标识号、标题、缩略词语、版本号、发行号）、系统概述、文档概述等方面的内容。

（2）引用文件。列出数据需求说明引用的所有文档的编号、标题、修改版本和日期。

（3）数据的逻辑描述。数据根据其逻辑属性的不同，可以分为动态数据和静态数据。静态数据是指在运行过程中主要作为参考的数据，它们在很长的一段时间内不会发生变化，一般不随运行而改变；动态数据包括所有在运行中要发生变化的数据，以及在运行中要输入、输出的数据。

（4）数据的采集。这一部分具体展开说明数据如何获取，主要包括要求和范围、数据输入的承担者、预处理、影响等。

（5）注解。包含有助于理解数据需求说明的一般信息。

（6）附录。提供那些为便于维护数据需求说明而单独编排的信息。

8.9.3 技术报告

作为项目的主要技术骨干，系统分析师经常需要编制各种体裁和用途的技术报告，这些技术报告本身就是一个总结经验、共享技术研究成果、实现团队知识共享的有效手段，同时也经常成为项目向上级相关主管部门申报成果的有效途径。

编制技术报告的主要目的是将项目开发中的技术研究工作做一个总结性的汇报，以文档化的结果将其中的成果保存起来。技术报告通常包括以下几个部分：

（1）引言。对技术报告进行概要性的描述，说明技术报告的编写目的、项目背景，并且对其中所使用的专业术语、缩略语建立专门的词汇解释表。

（2）项目解决的技术问题。这一部分主要是针对项目中解决了的技术问题进行描述，通常可以分为两部分。第一部分是项目的重要技术成果，也就是在项目中最重要的技术研究突破，应该对成果进行规范化描述；第二部分是主要技术指标的说明，对于项目中各项技术指标，例如，稳定性、可靠性、安全性、响应时间等方面，进行详细的说明与解释，并且主要应说明如何达到这些指标。

（3）系统的主要功能描述。概要性地描述软件系统的主要功能。

（4）在项目研发过程中遇到的技术问题和解决方法。这是技术研究报告中最重要的部分，也是最有价值的部分。在这一部分中，应该将每个技术问题作为一个单独的小节，在每个小节里充分地描述技术问题的现象、影响、发生环境等，然后进行充分的分析，尽量将导出解决方

法的思路写出来。这样才能够为下次遇到类似问题提供良好的支持。

(5) 系统目前达到的水平、现状、存在的问题及解决方案。第(4)部分是对已经解决的问题进行描述,而这一部分则是在前面的基础上进行提升。总结性地评价系统目前达到的水平,并对该系统的现状进行分析,指出还存在的问题,以及预期的解决方案,并说明现有系统未能有效解决问题的原因与限制。

(6) 项目总结。最后,在以上所有信息的基础上,做出客观、有效、总结性的描述,以便有关人员能够更好地了解项目的总体情况。

(7) 注解。包含有助于理解技术报告的一般信息,例如,背景信息、词汇表、原理等。这一部分应包含理解技术报告所需要的术语和定义,所有缩略词和它们在技术报告中的含义的字母序列表。

(8) 附录。提供那些为便于维护技术报告而单独编排的信息(例如,图表、分类数据等)。为便于处理,附录可以单独装订成册,按字母顺序编排。

8.9.4 项目开发总结报告

项目开发总结报告的编制,是为了总结项目开发工作的经验,说明实际取得的开发结果,以及对整个开发工作各方面的评价。根据 GB/T 8567—2006 的规定,项目开发总结报告应该包括以下几部分内容:

(1) 引言。包括项目开发总结报告的标识(包括标识号、标题、缩略词语、版本号、发行号)、系统概述、文档概述等方面的内容。

(2) 引用文件。列出项目开发总结报告引用的所有文档的编号、标题、修改版本和日期。

(3) 实际开发结果。对已经完成的工作进行概要性的总结,通常可以包括以下部分:产品、主要功能和性能、基本流程、进度、费用等。

(4) 开发工作评价。针对开发工作的情况进行简要性的评价,主要包括以下几个方面:对生产效率的评价、对产品质量的评价、对技术方法使用情况的评价、出错原因分析、风险管理等。

(5) 缺陷与处理。分别列出在需求评审阶段、设计评审阶段、代码测试阶段、系统测试阶段和验收测试阶段发生的缺陷及处理情况。

(6) 经验与教训。这是项目开发总结报告中最有价值的部分,列出从本次开发中获得的主要经验与教训,以及对今后的项目开发工作的建议。

(7) 注解。包含有助于理解项目开发总结报告的一般信息,例如,背景信息、词汇表、原理等。这一部分应包含理解项目开发总结报告所需要的术语和定义,所有缩略词和它们在项目开发总结报告中的含义的字母序列表。

(8) 附录。提供那些为便于维护项目开发总结报告而单独编排的信息(例如,图表、分类数据等)。为便于处理,附录可以单独装订成册,按字母顺序编排。